

Kahn's Algorithm

∴ Topological sort

⇒ Directed Graph, Dependency...

How? 값이 하나도 없는 (indegree = 0) 부터
찾아내고 값이 있는 노드의 indegree를 -1 하면서
찾아나갈 수 있다.

example :
 $A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow D$
 $C \rightarrow D$
→ or
 $A \rightarrow B \rightarrow C \rightarrow D$
or
 $A \rightarrow C \rightarrow B \rightarrow D$

Also, 사이클이 존재하는지 알아보아야
"Cycle"이 발생했는지나 판단.

Leetcode: course schedule II

build graph and count indegree

```
graph = defaultdict(list)
```

```
indegree = [0] * numCourses
```

```
for a, b in prerequisites:
```

```
    graph[b].append(a)
```

```
    indegree[a] += 1
```

Start with indegree 0 node

```
queue = deque()
```

```
for course in range(numCourses):
```

```
    if indegree[course] == 0:
```

```
        queue.append(course)
```

```
course_order = []
```

```
while queue:
```

```
    curr_course = queue.popleft()
```

```
    course_order.append(curr_course)
```

```
    for next_course in graph[curr_course]:
```

```
        indegree[next_course] -= 1
```

```
        if indegree[next_course] == 0:
```

```
            queue.append(next_course)
```

```
return course_order if len(course_order) == numCourses  
else []
```